

if(kakao)2022

카카오톡 메시징 시스템 재건축 이야기

C++ 서버를 Java 및 Kotlin으로

남기영 sirius.nam
카카오

카카오톡 메시징 시스템

10년 뒤를 내다보기

진행 과정

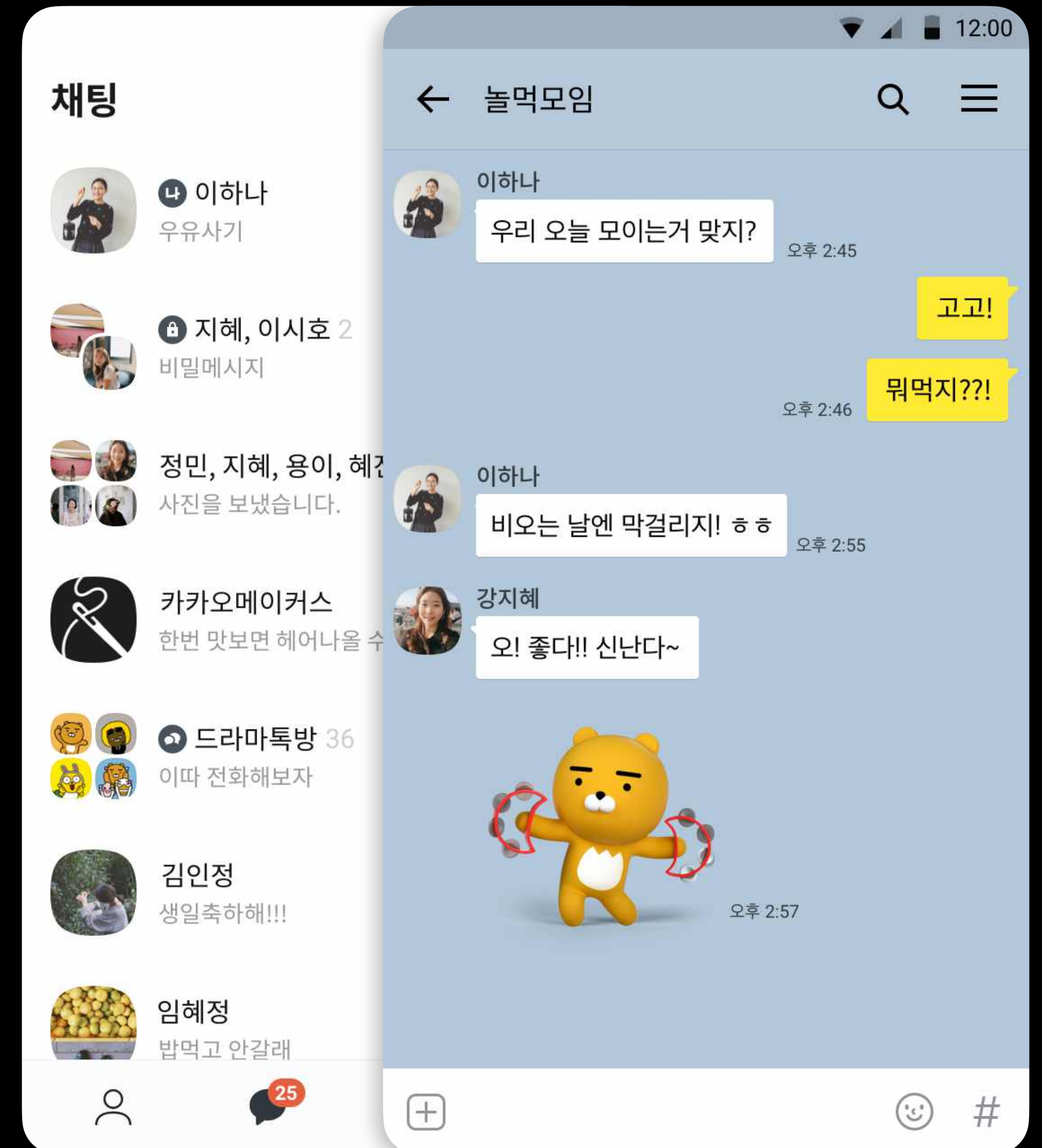
정리

앞으로의 계획

카카오톡 메시징 시스템

톡 메시징 파트

- 메시지 / 미디어 전송
- 채팅방 관리
- 메시지 관리 및 영구저장 (독서랍)
- 말풍선 관련 기능 개발 (멘션, 좋아요 등)



500K

일 평균 트래픽 (tps)

40M

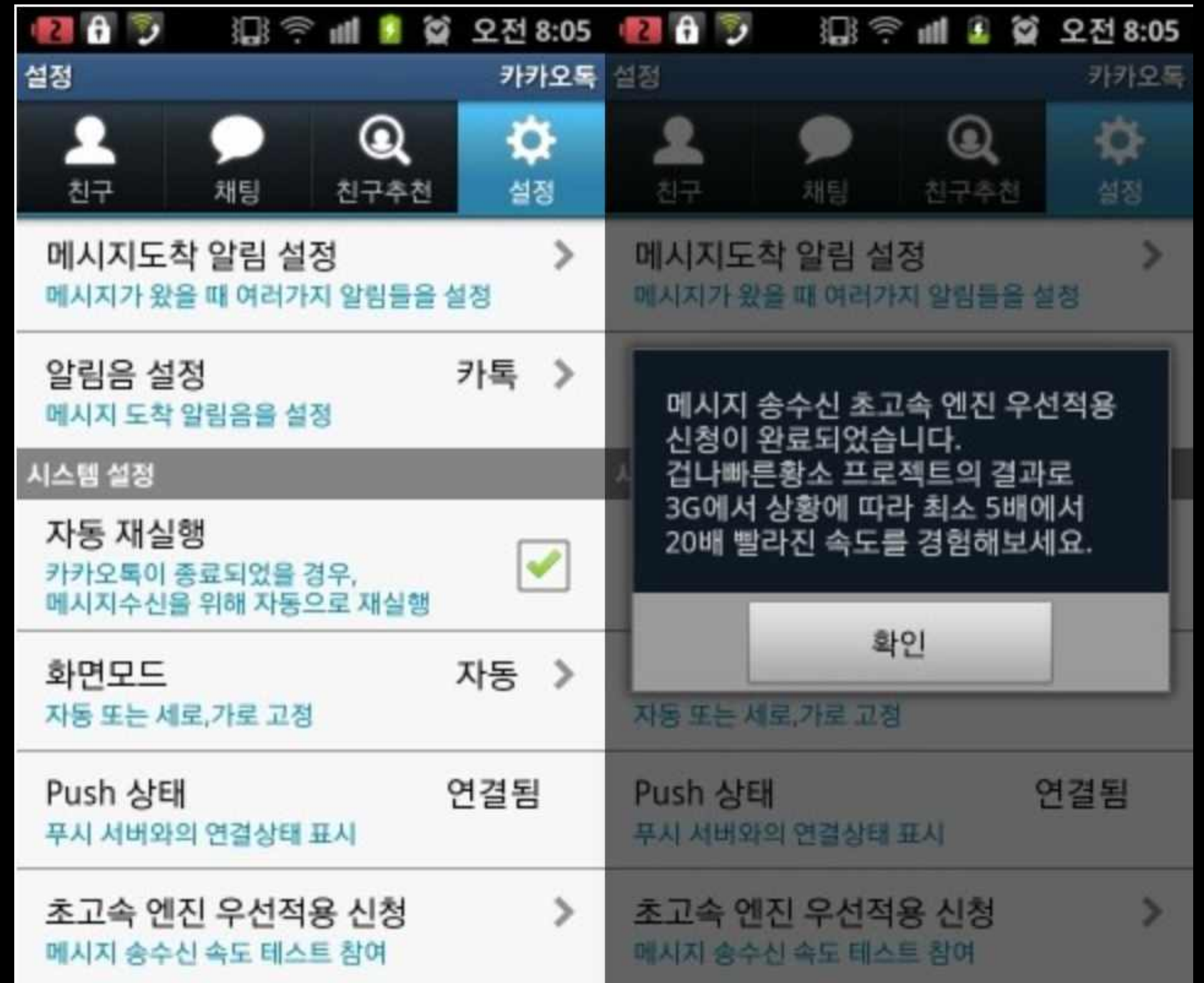
평균 연결 세션 수

6.5M

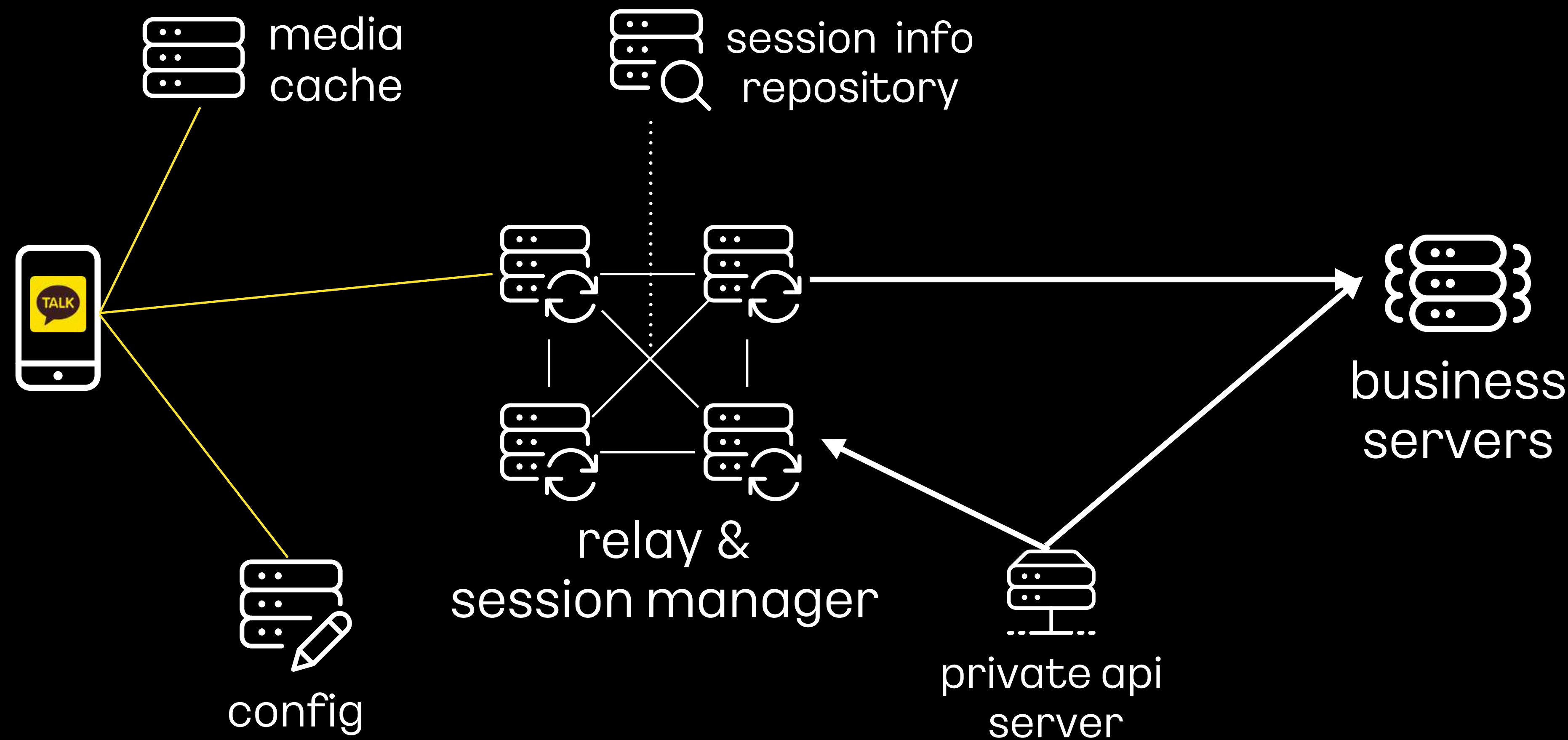
최고 트래픽 (tps)

겁나 빠른 황소 프로젝트 (2011년)

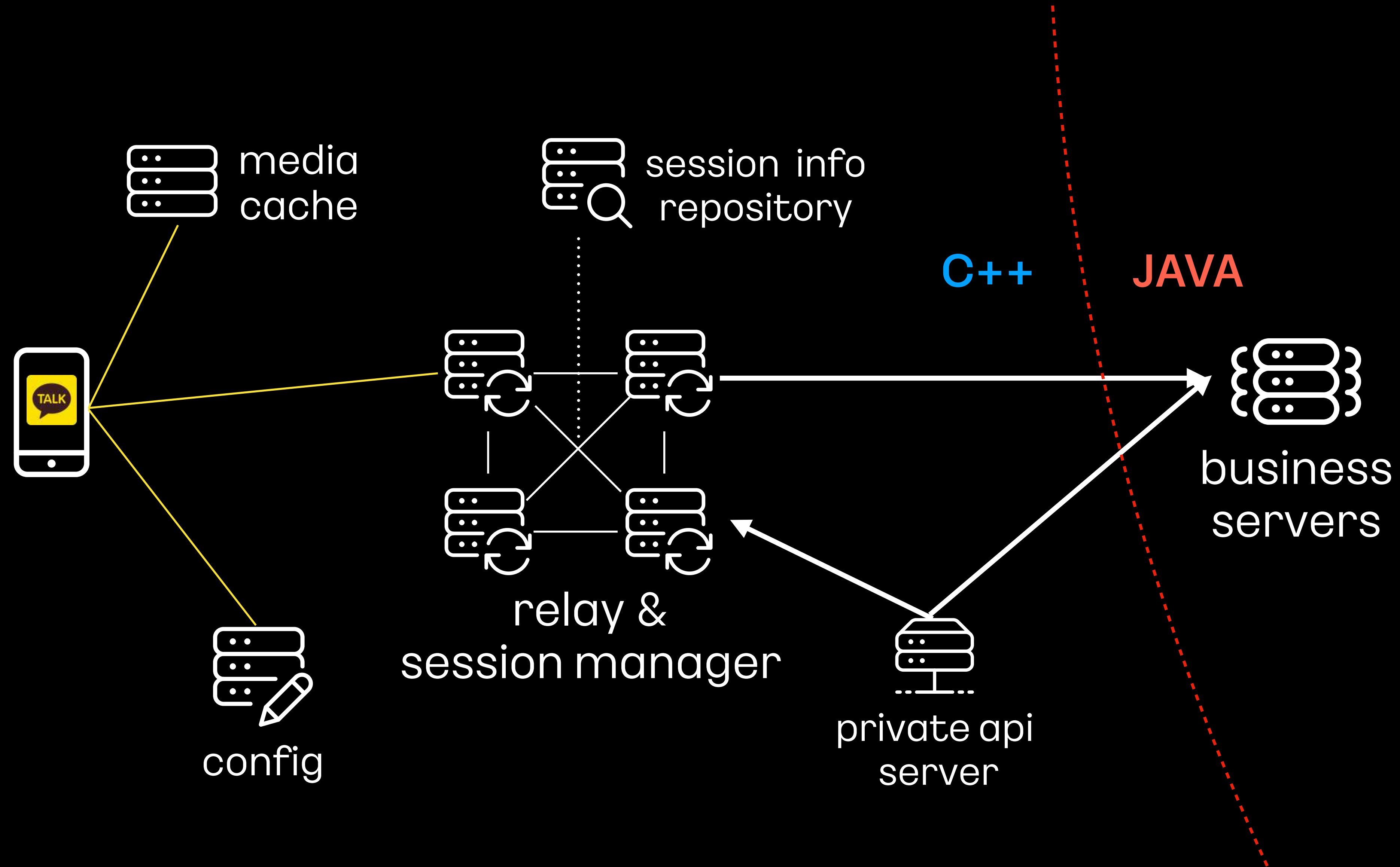
- 자체 프로토콜을 통한 패킷 경량화 및 통신 최적화
- 푸시 시스템 구조 최적화
- 백엔드 시스템 성능 개선



메시징 백엔드 시스템 구조



메시징 백엔드 시스템 구조

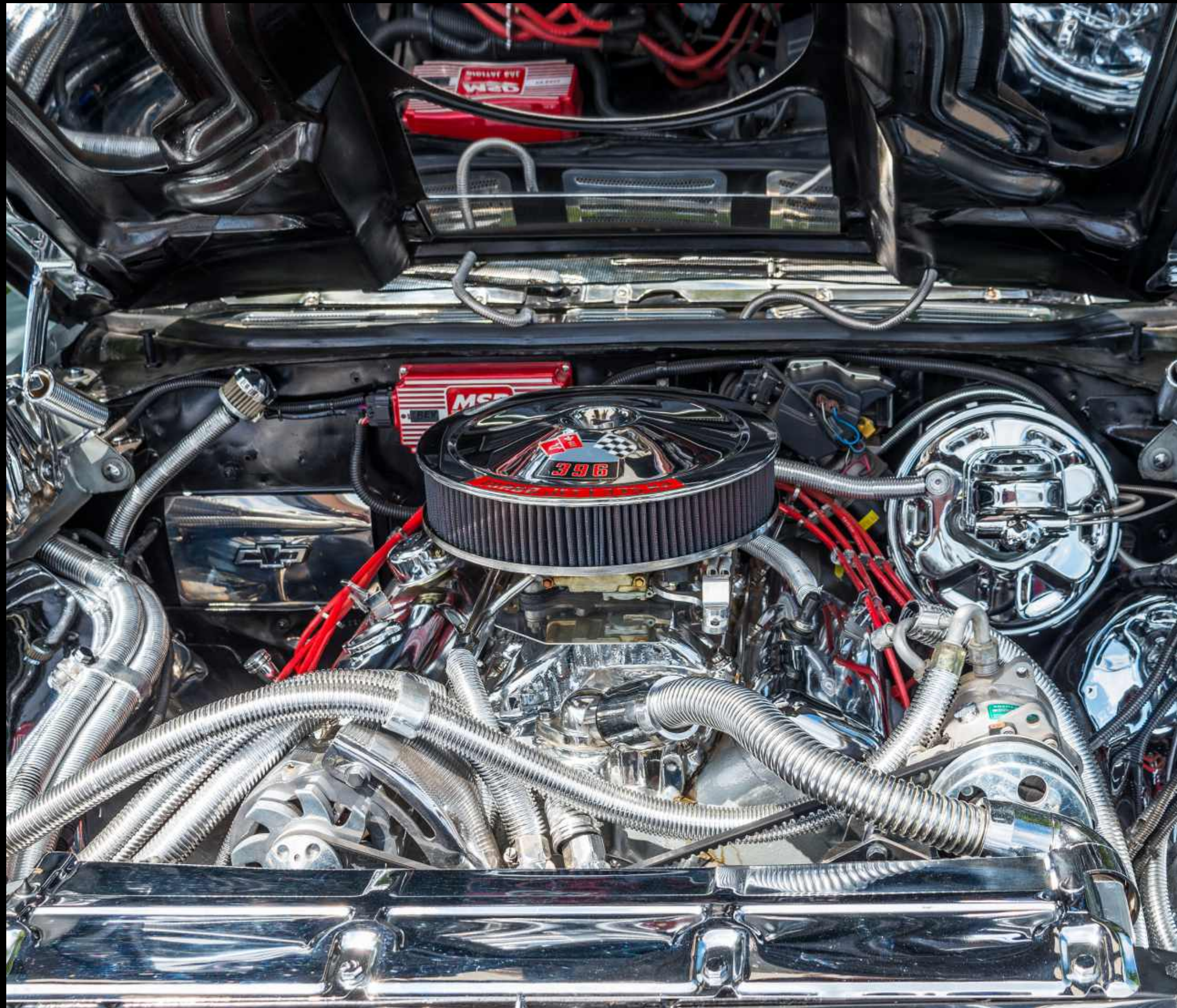


C++ 백엔드 서버 어플리케이션

- epoll 기반 비동기 입출력
- 스레드 별로 미리 할당한 메모리 버퍼 사용
- 대당 500K 이상 세션 관리

10년 뒤를 내다보기

커스텀

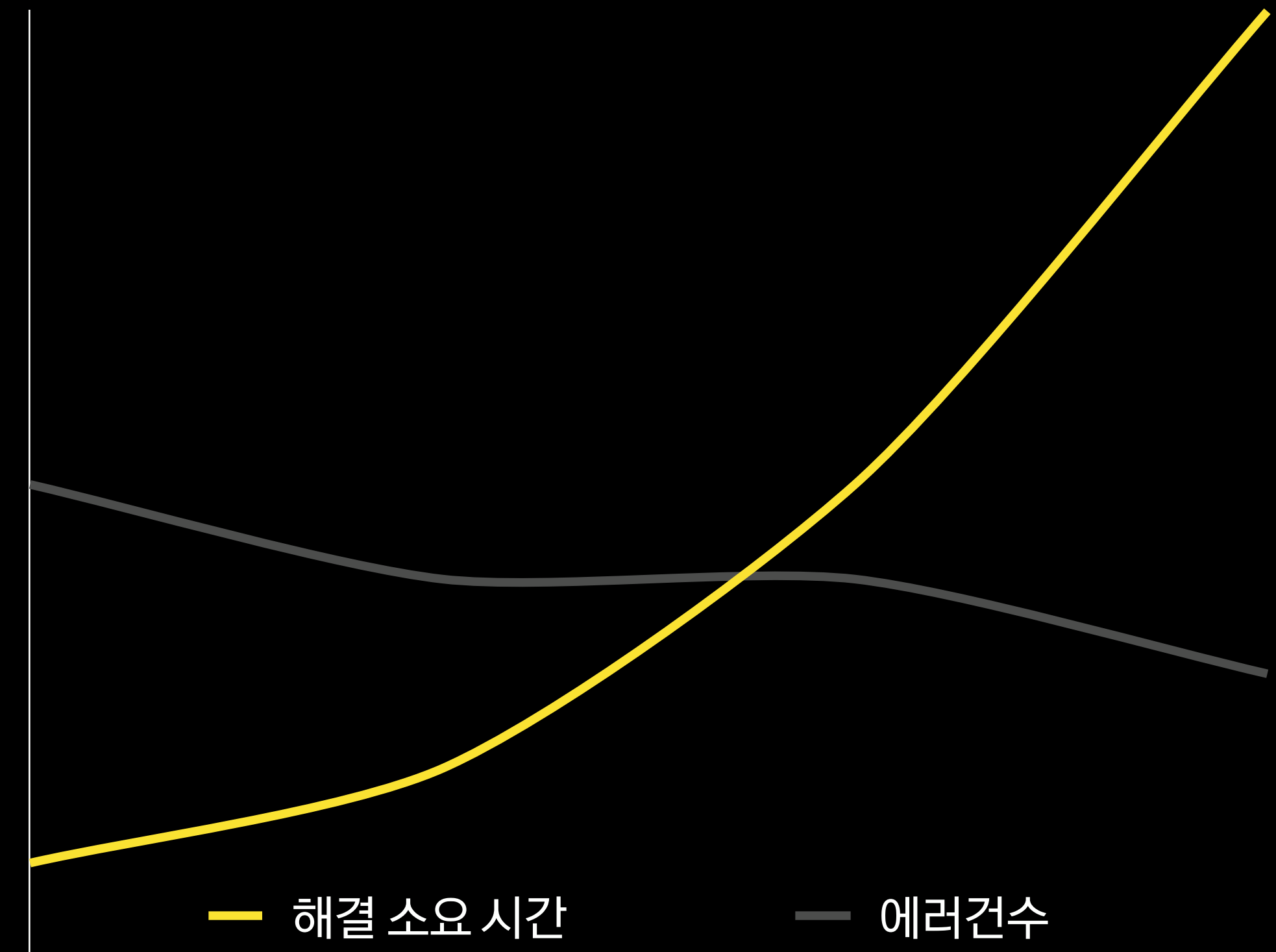


양산



경직된 코드

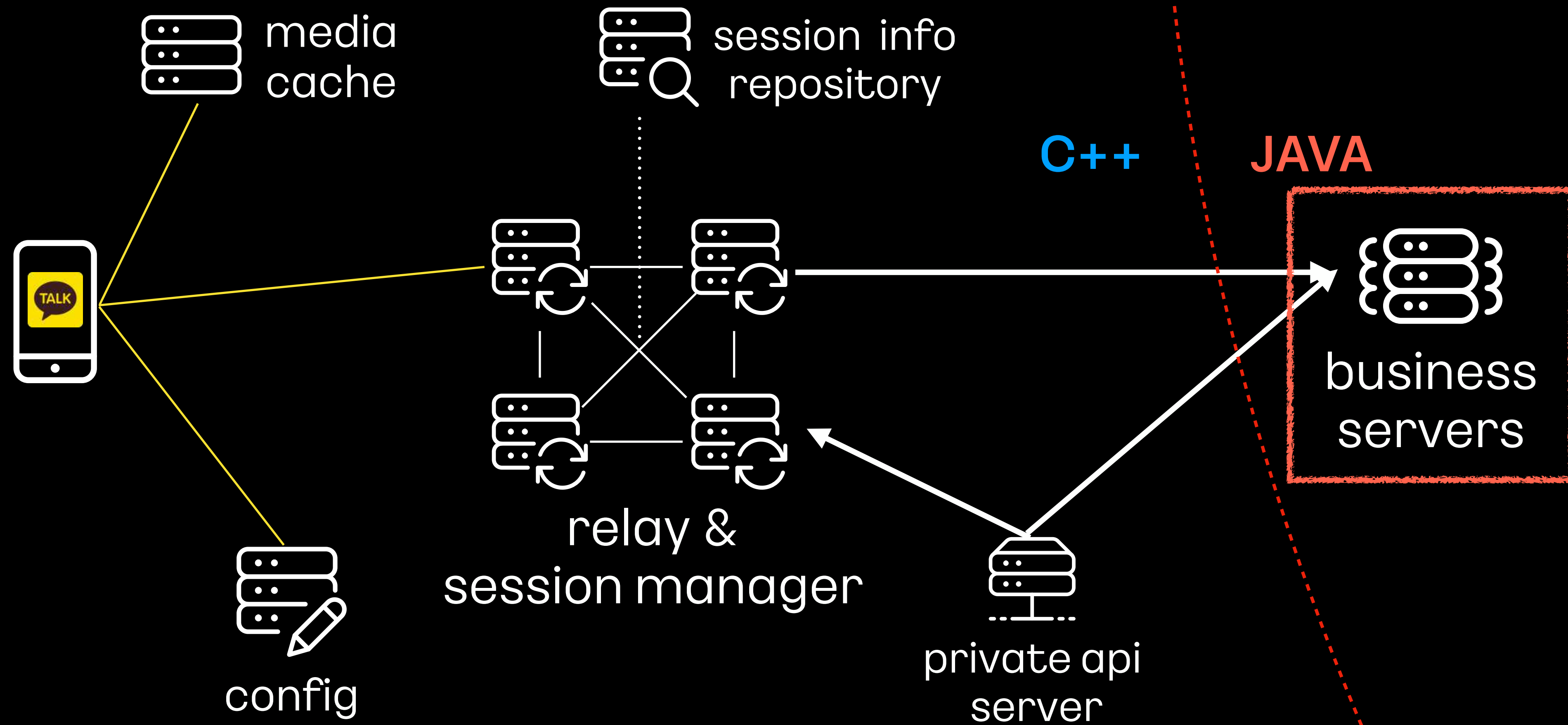
- 계층간 강결합: Transport / Application
- 수많은 매크로, 최적화 코드로 인한 가독성 저하
- 제약적 외부 라이브러리 사용
- 안정적, 하지만 치명적으로 다가오는 메모리 버그



어려운 유지보수

- 적은 단위 테스트, 너무 복잡한 통합 테스트
- 오래된 배포 시스템 : shell script & python
- 파트내 인적 리소스 불균형: C++ / JAVA
- 10년 후에도 문제없이 유지보수가 가능할까?

옆 동네는 어떻게 하고있더라?



Business servers

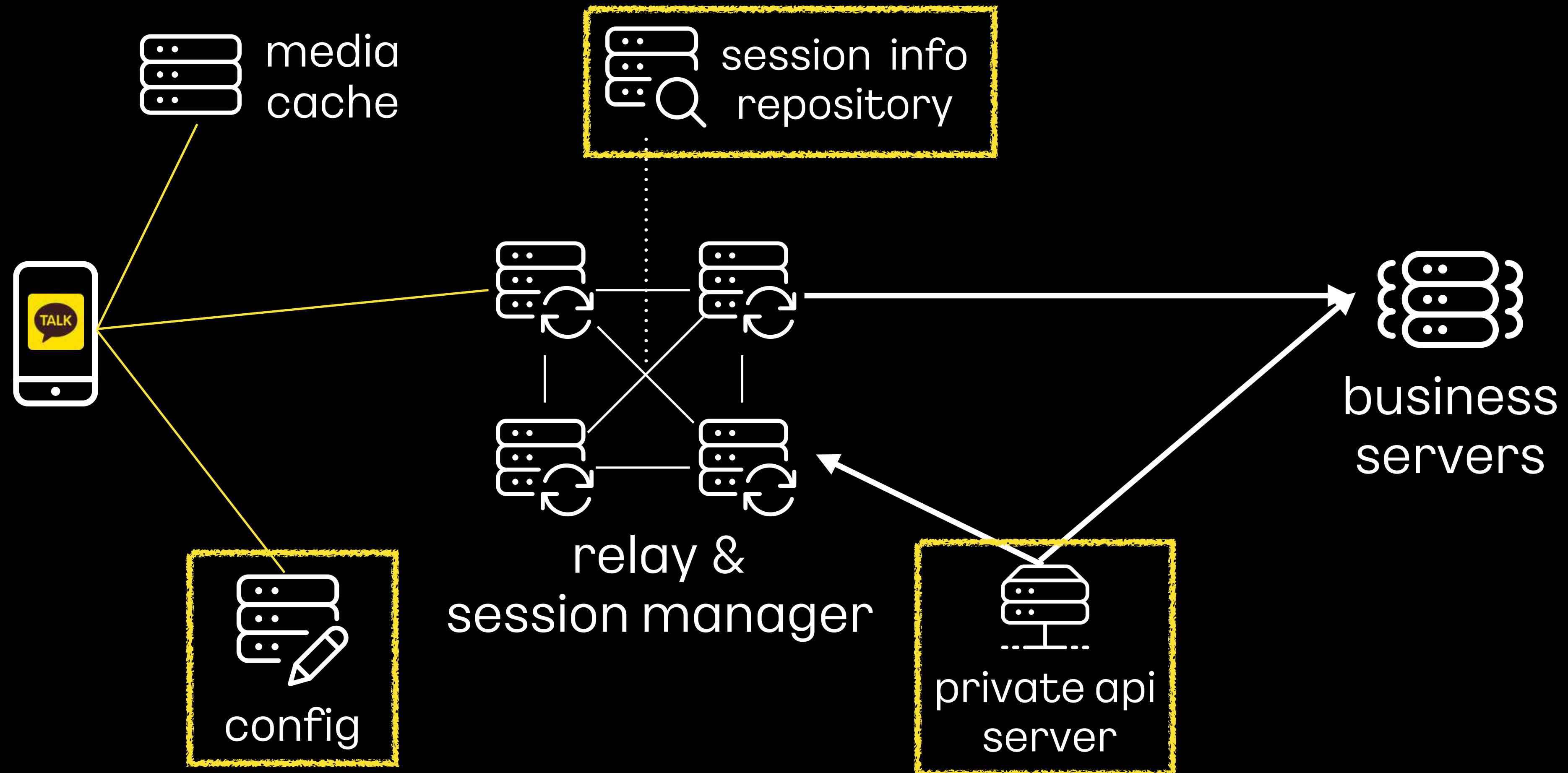
- Java
- Spring framework
- Netty framework with epoll
- Jenkins
- BDD with cucumber^[1]
- JUnit

[1] 카카오톡 시스템의 진화 - 테스트 자동화로 자신감 갖기, 2020 if kakao, tomo.213

재건축 - 1단계

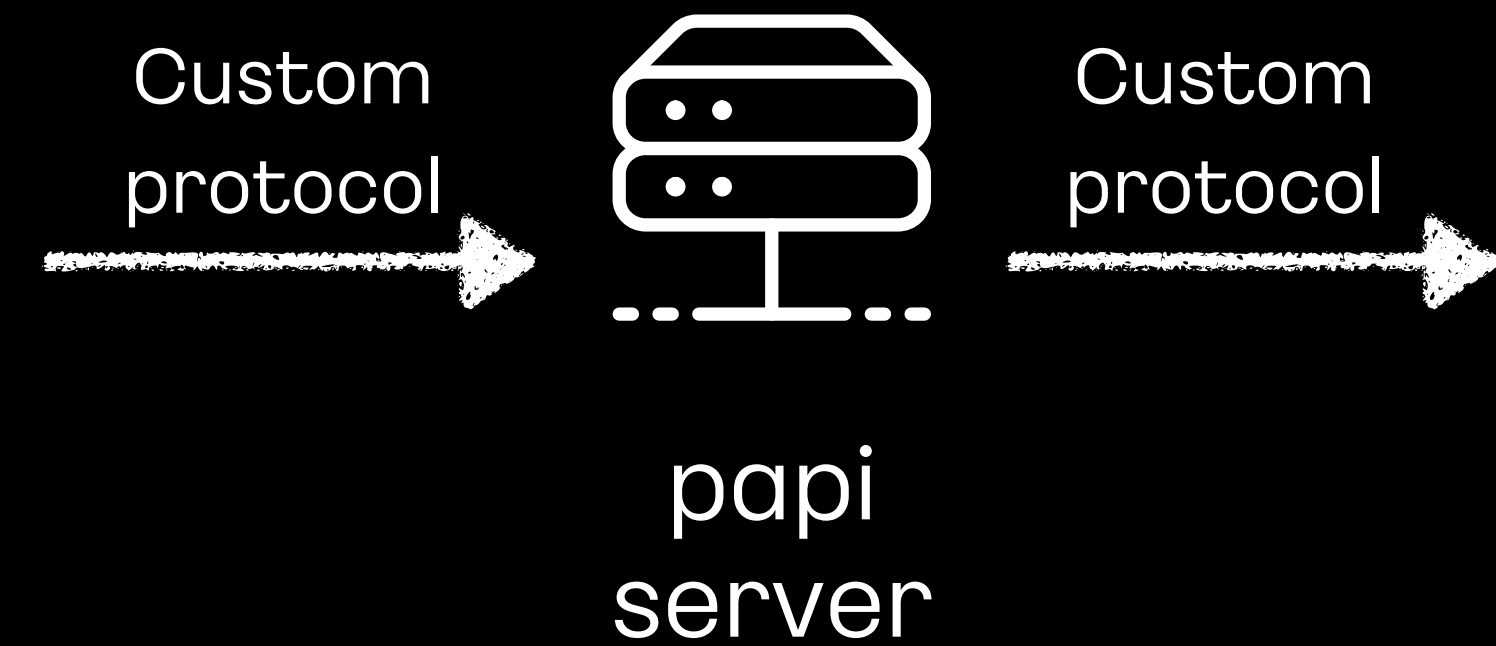
재건축 1차

- 트래픽이 비교적 적고, 장애 영향도가 낮은 서비스 부터



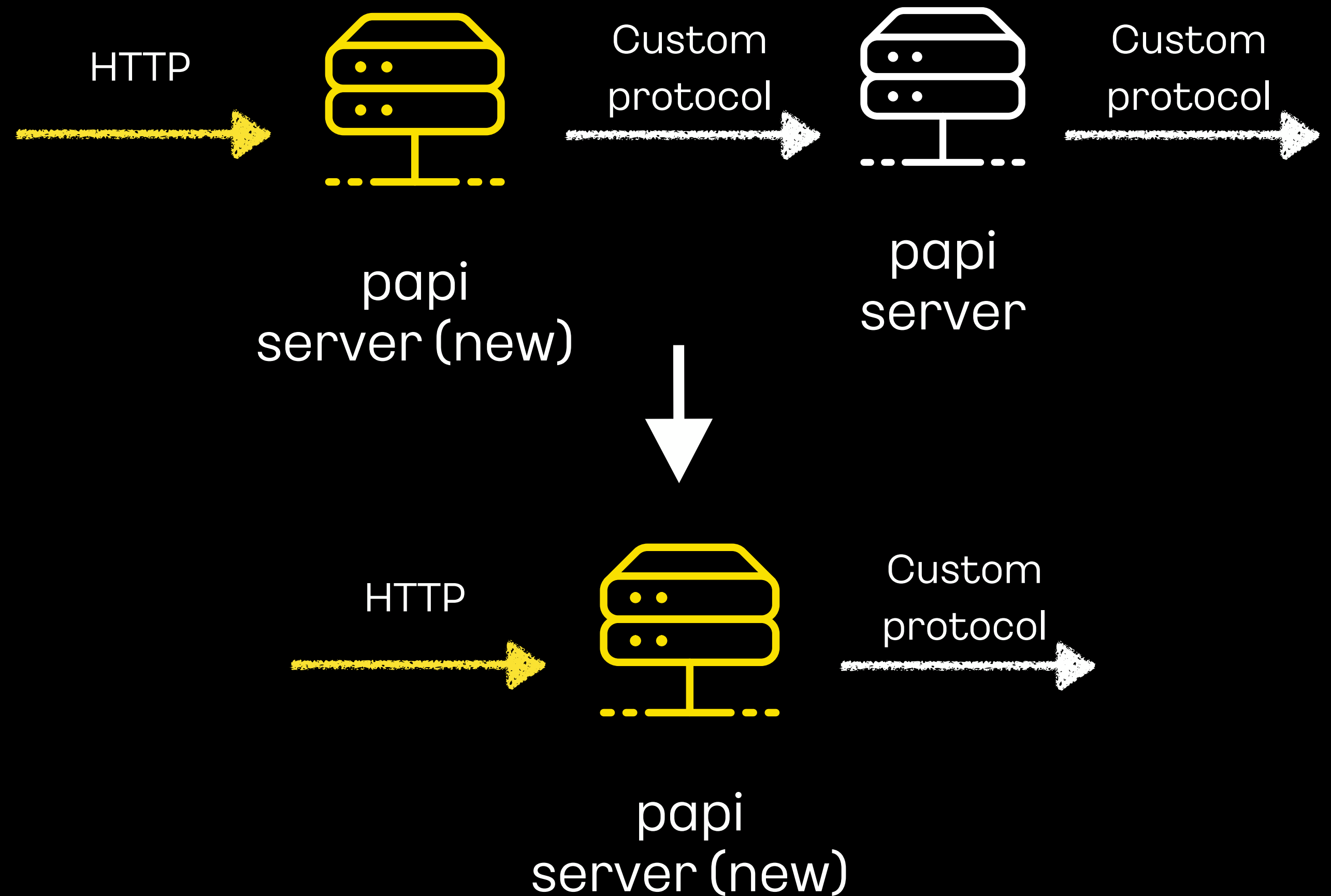
Private api server: As-Is

- 사내 서비스를 위한 api 서버
- 호출시 자체 메시징 프로토콜 지원 필요
- 문서화 부족



Private api server: To-Be

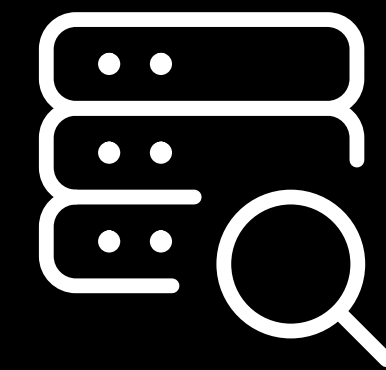
- HTTP 호출 지원
- AsciiDoc을 이용한 문서화
- k8s를 이용한 유연한 배포 및 확장성 증대^[1]



[1] 카카오톡 적용 사례를 통해 살펴보는 카카오 클라우드의 Kubernetes as a Service, 2019 if kakao, greg.47

Session info repository : As-Is

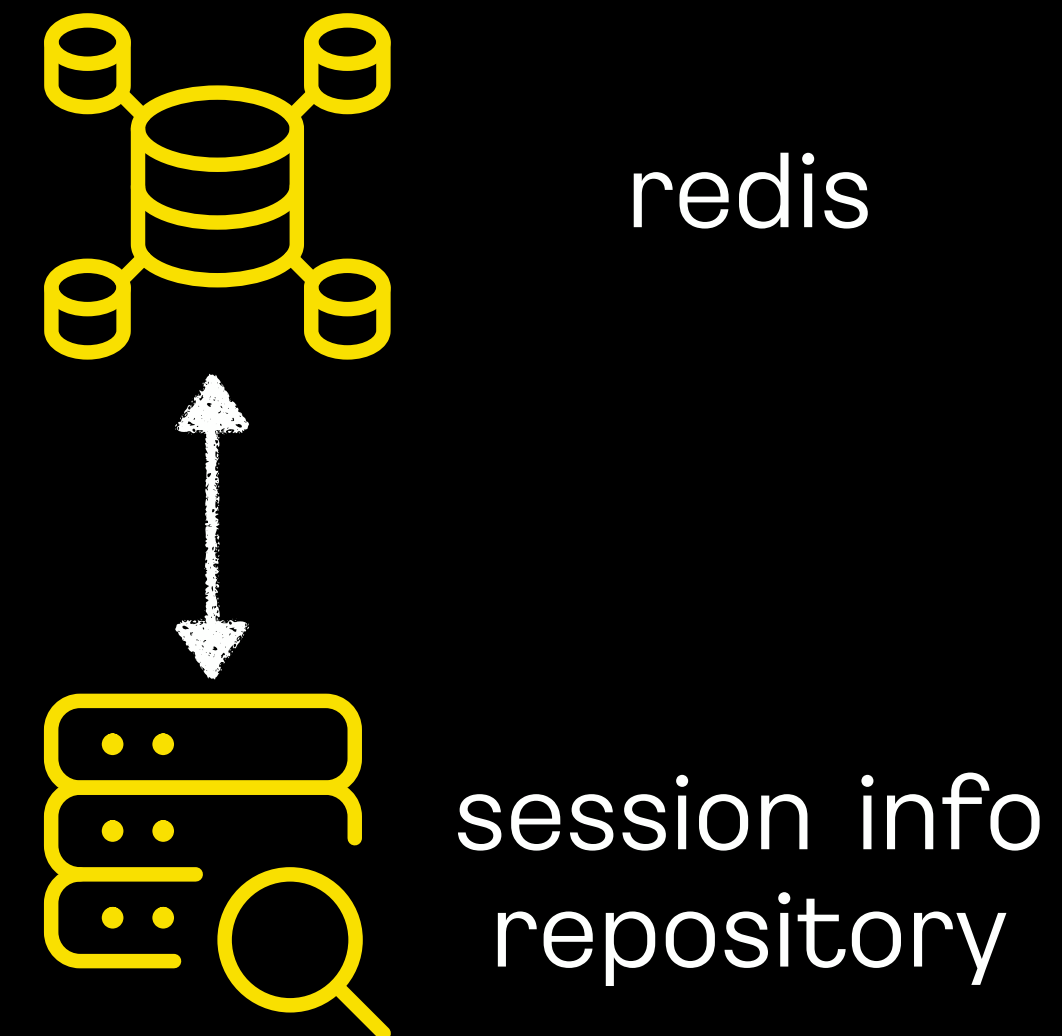
- 세션 정보 저장 서버
- In-memory 저장
- 확장 및 기능추가의 어려움



session info
repository

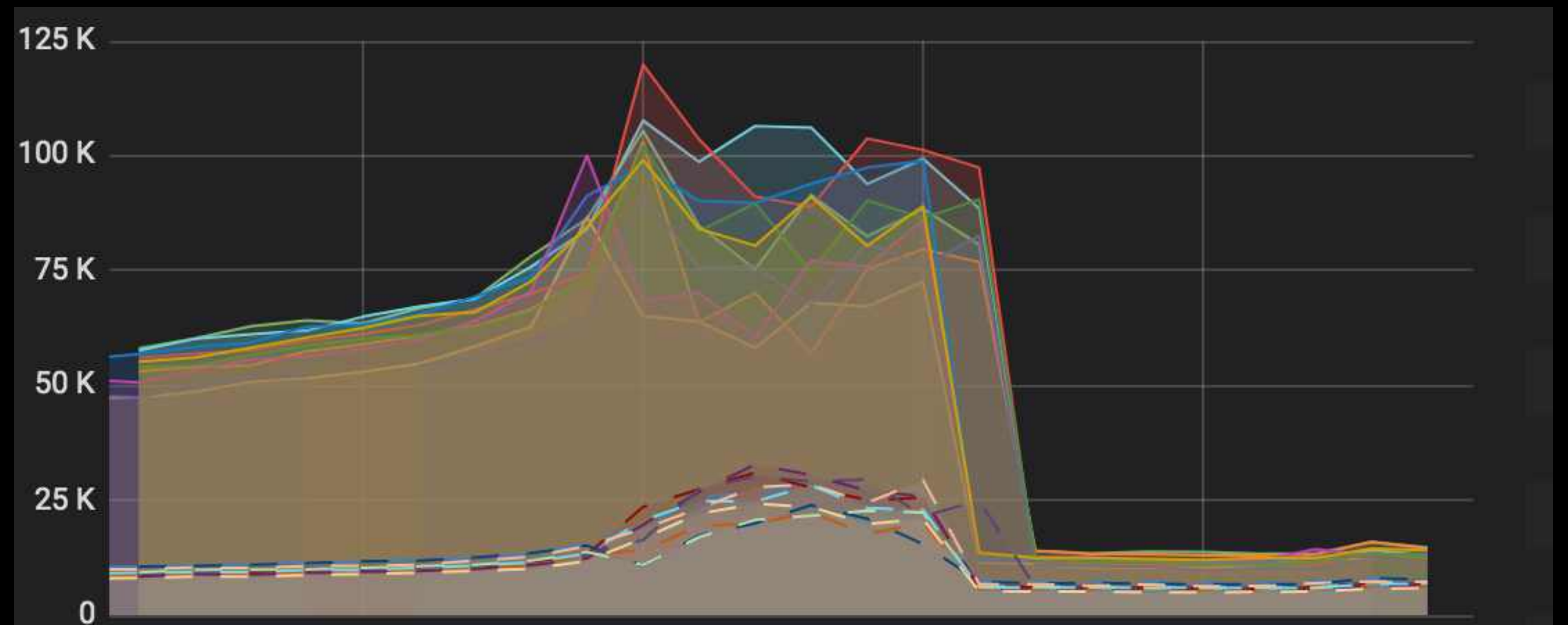
Session info repository : To-Be

- 저장소 분리: Redis (cluster)
- 고도화된 세션 정보 저장 가능 (ex: 읽지 않은 메시지 개수 등)
- k8s를 통한 유연한 배포 및 확장

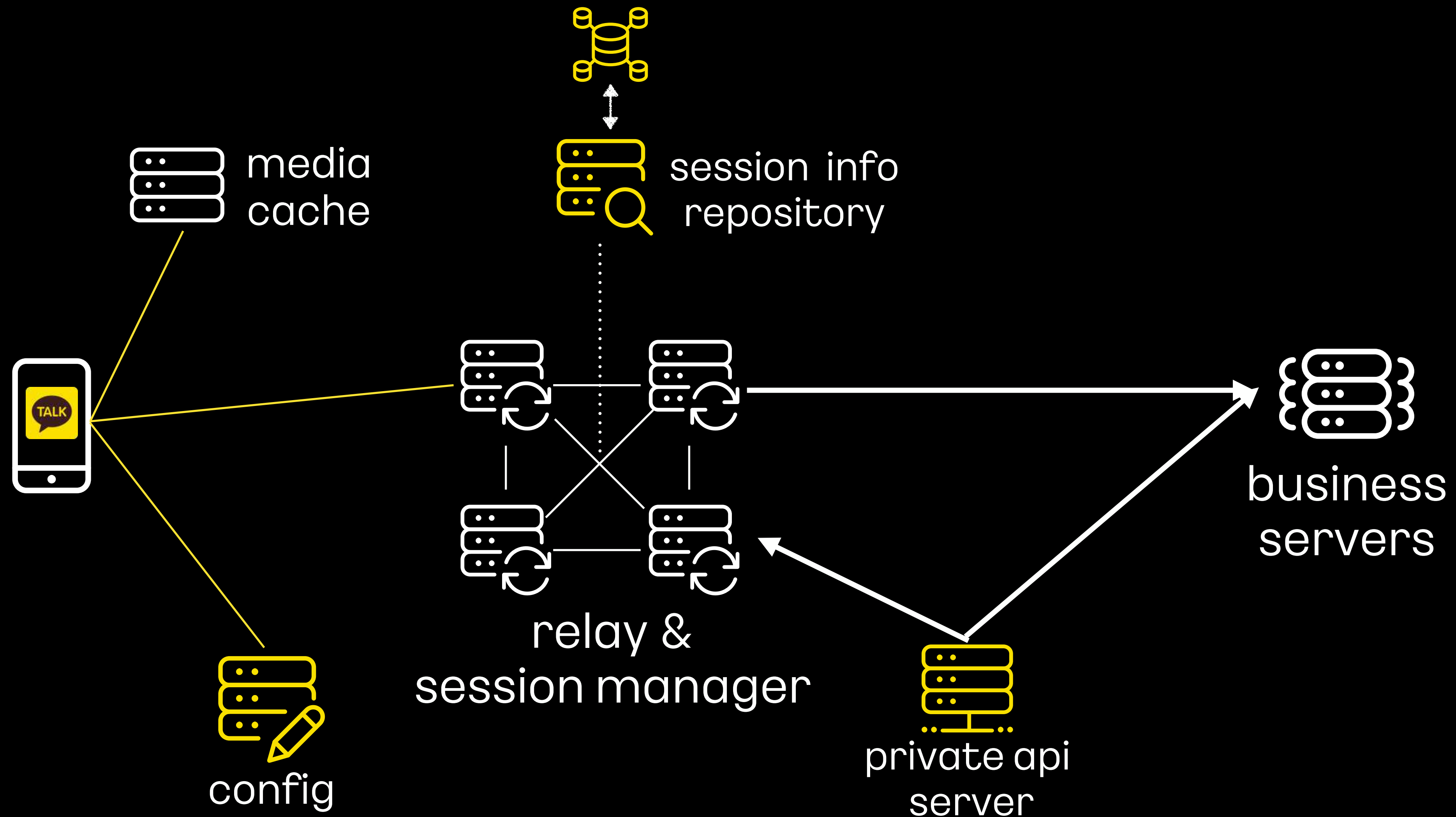


Session info repository : Redis

- 적용 중 레디스 피크 트래픽 산정 실패
- 각 레디스 버킷당 100K tps 이상 넘지 않도록 설정
- Redis client 변경: Jedis → Lettuce



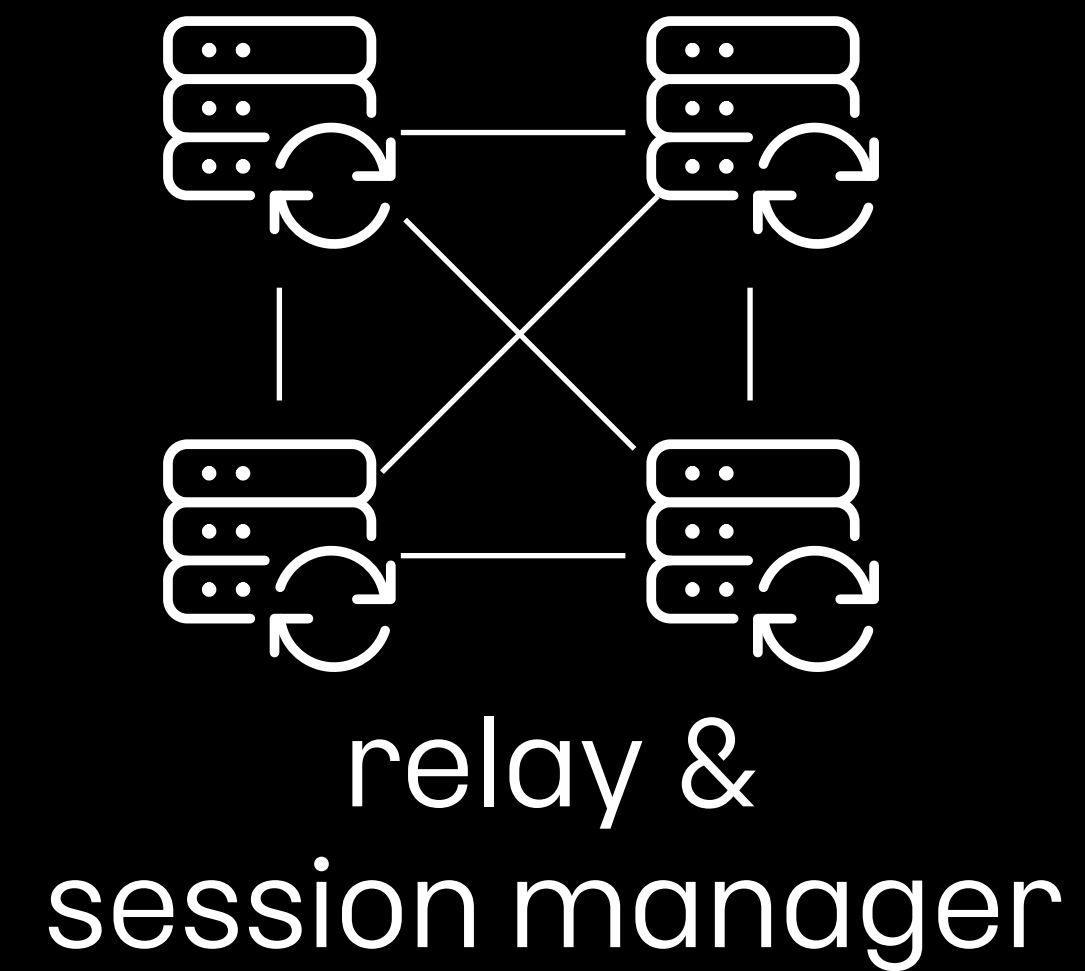
재건축 1차 결과



재건축 - 2단계

Relay & session manager

- 500K sessions
- 15~30K tps, max 100K tps



Relay & session manager: 고민점

- GC로 인한 stop-the-world 영향 최소화 : ZGC
- 빠르고 편한 개발: Kotlin^[1]
- 비동기 릴레이: Thread pool executor, 추후 coroutine
- Full-mesh 에서 한 대씩 신규 서버 적용: 장애 위험성 고려
 - 클라이언트 / 타 서버에서는 블랙박스
- 기존 서버 대비 성능 이슈
 - 잦은 메모리 할당으로 인한 성능저하가 있을까?



[1] 카카오톡의 서버사이드 코틀린, 2018 if kakao, indy.jones

Benchmark

	C++ server	kotlin server (g1gc)	Kotlin server (zgc)
CPU load (LOGIN)	0.6	1.6	1.6
CPU load (WRITE)	0.9	1	1
Memory	13 GB	13 GB	13 GB
Response time (WRITE, avg)	6 ms	5 ms	5 ms
Relay time (avg)	47 ms	42 ms	42 ms
Maximum pause gc time	-	170ms	1ms

Intel Xeon 3.6GHz, 16GB Ram

Benchmark : 프로토콜 처리 안정성

g1gc

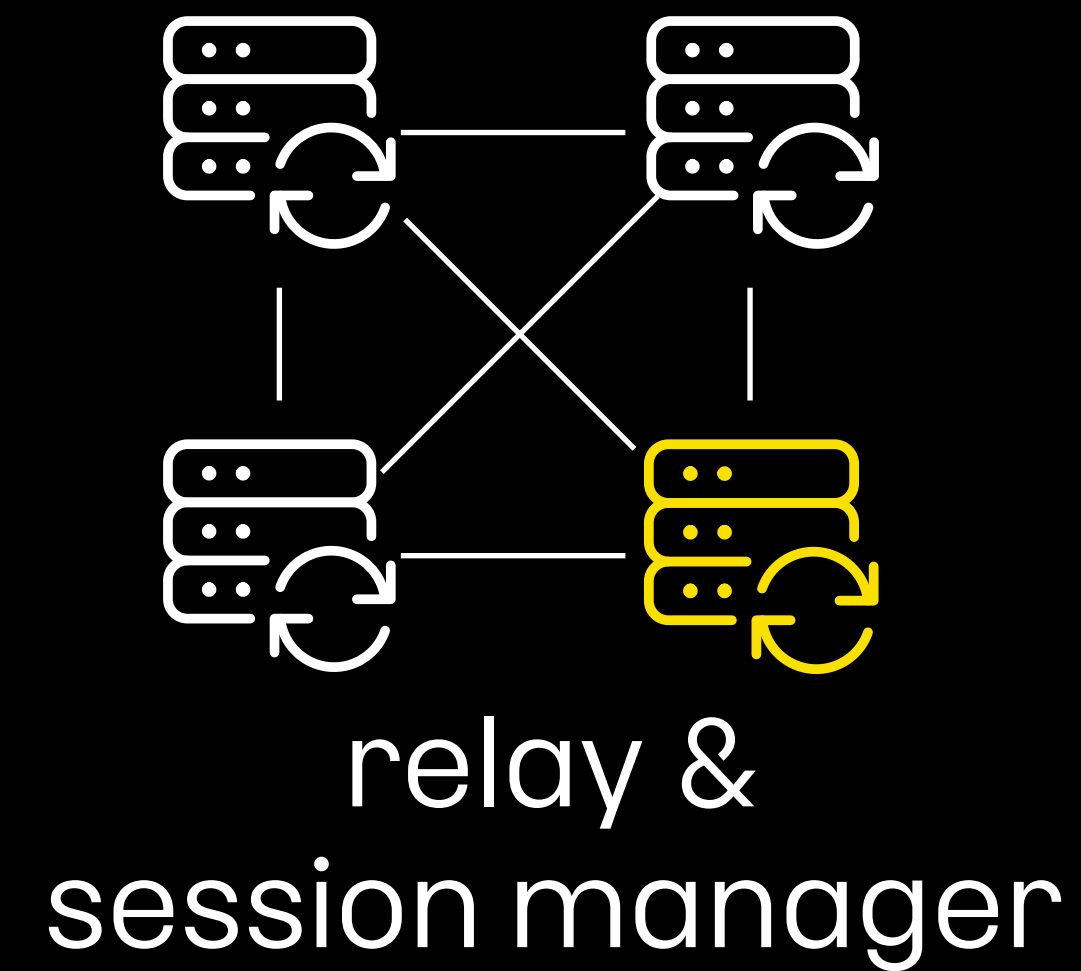


zgc

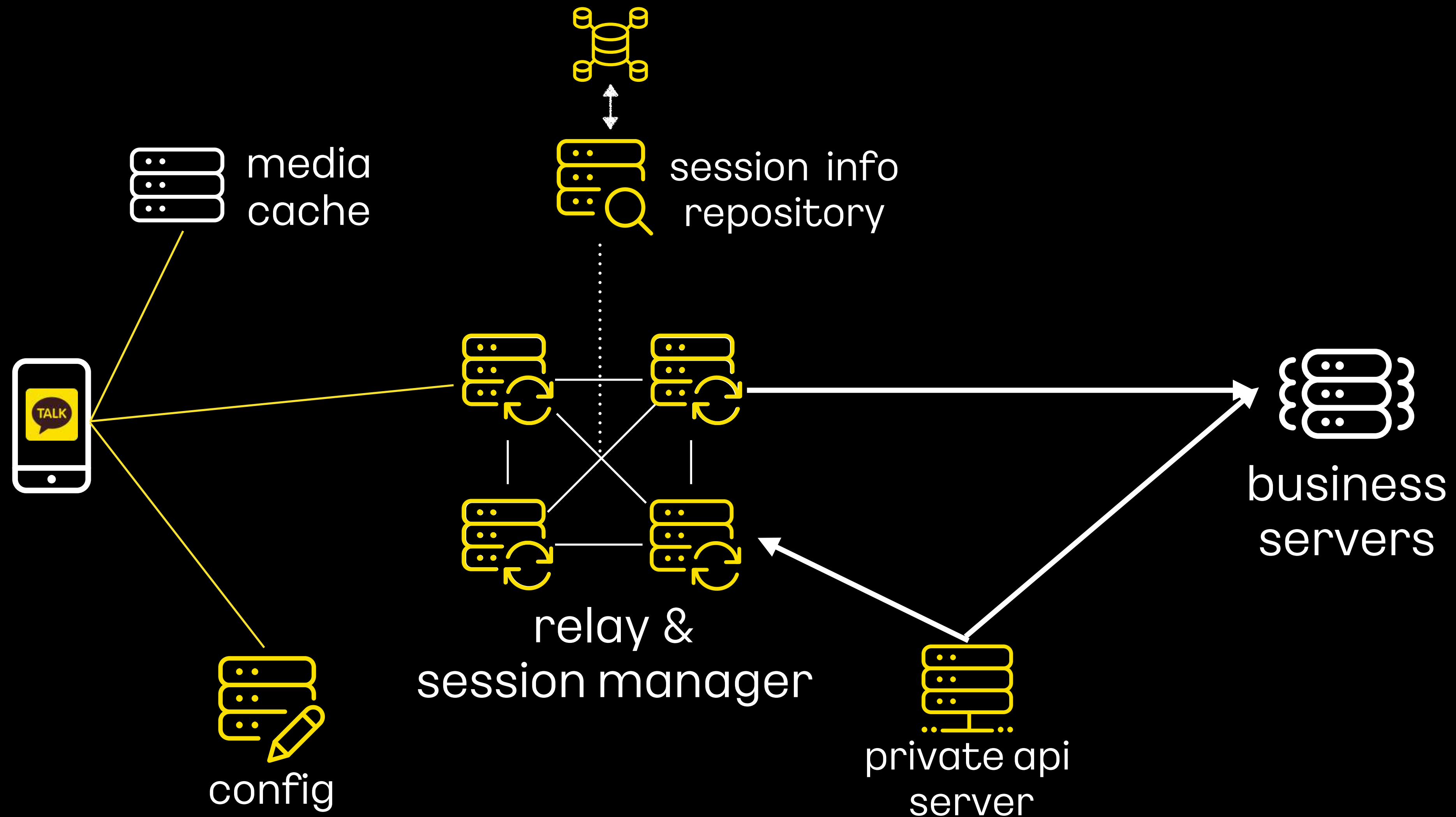


최적화 작업 (진행중)

- 멀티 디바이스를 위한 세션 구조 개편
- 동시성 영향 받는 데이터들의 Lock 구조 개선
- GC 설정 튜닝
- 비동기 릴레이: thread pool, 이후 coroutine



재건축 2차 결과 (진행중)



정리

	As-Is	To-Be
Deploy	Shell & python script	Jenkins, k8s (with argoCD)
Unit test	gTest	JUnit
Integration test	Python script (remote stage)	Cucumber (local stage, dockerize)
Transport layer	Integrated with application	Layered (with netty pipeline)

개선사항

- 간결해진 코드: 절반 이상 감소한 코드라인
- 접근성이 높아진 배포 및 운영: 터미널 → 웹
- 가용 인력 3배 증가
- 실수를 줄여주는 직관적인 통합 테스트
- 계층 분리를 통해 유연해진 디자인

재건축, 그 다음은?

- 내부 프로토콜 구조 개선
- 대규모 채팅방 릴레이 로직 개선
- 미디어 캐시 개선

감사합니다